Exam Practice: Indexing Structures for Files and Physical Database Design

Topic Coverage: Single-level ordered indexes (primary, clustering, secondary) · Multilevel indexes & ISAM · B-trees and B+-trees · Multiple-key access (composite indexes, partitioned hashing, grid files) · Hash indexes · Bitmap indexes · Function-based indexing · Logical vs. physical indexes · Physical database design factors and decisions

Format: 20 MCQ (1 mark each = 20 marks) · Structured Questions (40 marks) · Essay Questions (40 marks)
Total: 100 marks | Suggested time: 3 hours

Section A — Multiple Choice Questions (20 Marks)

Choose the best answer for each question. 1 mark each.

1. An index structure that has an index entry for every search key value (and hence every record) in the data file is called:
A. A sparse index
B. A dense index
C. A clustering index
D. A multilevel index

2. A primary index can only be created on a field that is:
A. Any nonkey field
B. The ordering key field of the file
C. A foreign key
D. A composite field

3. Why can a data file have at most one primary index OR one clustering index, but never both?
A. DBMS license restrictions limit index counts
B. A file can have at most one physical ordering field
C. Both index types require the same storage format
D. Clustering indexes are deprecated in modern DBMSs

4. A clustering index is built on:
A. A key field used for physical ordering
B. A nonkey field used for physical ordering
C. Any nonordering field
D. The primary key only

5. Which type of single-level index is classified as dense when built on a key (unique) field?
A. Primary index
B. Clustering index
C. Secondary index
D. None of the above

6. In multilevel indexing, the blocking factor of the index (bfri) is also referred to as the:
A. Search radius
B. Fan-out (fo)
C. Occupancy ratio
D. Anchor factor

7. Searching a multilevel index with fan-out *fo* and *bi* first-level blocks requires approximately how many block accesses (excluding the final data block access)?
A. $\log_2(bi)$
B. bi / 2
C. log_fo(bi)
D. fo × bi

8. In a B-tree of order *p*, each node (except root and leaf) must have at least how many tree pointers?
A. p
B. p − 1
C. ⌈p/2⌉
D. 2p

9. What is the key structural difference between a B-tree and a B+-tree?
A. B-trees are unbalanced; B+-trees are always balanced
B. In a B+-tree, data pointers appear only at leaf nodes, while in a B-tree, data pointers appear at every level
C. B+-trees do not support range queries
D. B-trees can only be built on key fields

10. Why are B+-trees generally preferred over B-trees as access structures for database files?
A. B+-trees are always shallower because internal nodes hold only keys and tree pointers (no data pointers), so they hold more keys per node and accommodate more entries for a given number of levels
B. B+-trees do not require balancing algorithms
C. B-trees cannot support deletion operations
D. B+-trees do not need leaf nodes

11. After numerous random insertions and deletions, B-tree and B+-tree nodes stabilize at approximately what occupancy level?
A. 50%
B. 69%
C. 85%
D. 100%

12. A search tree of order *p* has, at most, how many search values per node?
A. p
B. p − 1
C. p + 1
D. 2p

13. Partitioned hashing, as an extension of static external hashing for multiple keys, has which major limitation?
A. It cannot handle equality conditions
B. It cannot handle range queries

C. It requires a B+-tree to operate
D. It only works on single-attribute keys

14. In a grid file built on two attributes (e.g., Dno and Age), what does each cell of the grid array point to?
A. A B+-tree root node
B. A bucket address where matching records are stored
C. A hash function definition
D. Another grid array

15. A hash index is best described as:
A. A primary file organization that replaces ordered files entirely
B. A secondary access structure using hashing on a search key, separate from the field used for the primary file organization
C. Only usable with range queries
D. A structure that requires no index entries

16. Bitmap indexing is most efficient and most commonly applied to columns that have:
A. A very large number of distinct values, all unique
B. A small number of distinct values, relative to the number of rows
C. Only numeric data types
D. Only foreign key columns

17. Given a column where a particular value occurs very frequently in a B+-tree leaf node (nonkey search field), what storage optimization is suggested in place of storing many individual record pointers?
A. A grid file
B. A bitmap (bitvector)
C. Partitioned hashing
D. A logical index

18. What is the main advantage of a logical index <K, Kp> over a physical index <K, P>?
A. It is always smaller in storage size
B. It avoids the need to update pointers when the physical address of a record changes (e.g., due to bucket splitting in extendible hashing)
C. It eliminates the need for a primary file organization
D. It only works with B-trees

19. Function-based indexing (e.g., `CREATE INDEX upper_ix ON Employee (UPPER(Lname))`) is primarily useful because:
A. It removes the need for any WHERE clause in SQL
B. It allows the DBMS to use an index even when a function is applied to a column in the search predicate, avoiding a full table scan
C. It converts a B+-tree into a hash index automatically
D. It only works with the DELETE statement

20. According to the physical database design guidelines, which attributes are the best candidates for being **excluded** from indexing consideration to avoid unnecessary update overhead?
A. Candidate key attributes
B. Attributes frequently used in join conditions

C. Attributes whose values are frequently changed by update operations
D. Attributes used in range queries

Section A — Answer Key

Q	Answer	Q	Answer	Q	Answer	Q	Answer
1	B	6	B	11	B	16	B
2	B	7	C	12	B	17	B
3	B	8	C	13	B	18	B
4	B	9	B	14	B	19	B
5	C	10	A	15	B	20	C

Section B — Structured Questions (40 Marks)

Answer ALL questions in this section.

Question 1 (12 marks)

A file has **r = 30,000** fixed-length EMPLOYEE records on a disk with block size **B = 1,024 bytes**. The record size is **R = 100 bytes**, and the file is unspanned.

(a) Calculate the blocking factor (bfr) and the number of file blocks (b) for the data file. **(4 marks)**

(b) The file is ordered by the key field `Ssn`, which is **9 bytes** long, and a block pointer is **6 bytes** long. Calculate the size of each primary index entry, the index blocking factor (bfri), and the number of first-level index blocks (bi). **(4 marks)**

(c) Calculate the number of block accesses required to retrieve a record using (i) a binary search directly on the data file, and (ii) a binary search using the primary index plus one data block access. State which is more efficient and explain why in one sentence. **(4 marks)**

Model Answer — Question 1

(a)

- bfr = ⌊B/R⌋ = ⌊1,024/100⌋ = **10 records per block**
- b = ⌈r/bfr⌉ = ⌈30,000/10⌉ = **3,000 blocks**

(b)

- Index entry size Ri = (9 + 6) = **15 bytes**
- bfri = ⌊B/Ri⌋ = ⌊1,024/15⌋ = **68 entries per block**
- Number of index entries ri = number of data file blocks = 3,000
- bi = ⌈ri/bfri⌉ = ⌈3,000/68⌉ = **45 blocks**

(c)

- (i) Binary search on data file: $\lceil \log_2 b \rceil = \lceil \log_2 3{,}000 \rceil =$ **12 block accesses**
- (ii) Binary search on primary index: $\lceil \log_2 bi \rceil = \lceil \log_2 45 \rceil =$ **6 block accesses**, plus 1 data block access = **7 total block accesses**
- The primary index is more efficient (7 vs. 12 block accesses) because the index file is much smaller than the data file (fewer, smaller entries), so a binary search over it requires fewer block accesses, at the small added cost of one extra access to fetch the actual data block.

Question 2 (10 marks)

(a) Define the following terms, in your own words: *indexing field*, *block anchor*, *dense index*, *sparse (nondense) index*. **(8 marks — 2 marks each)**

(b) State whether each of the following index types is dense or sparse (nondense), and briefly justify each answer: (i) Primary index, (ii) Secondary index on a key field. **(2 marks)**

Model Answer — Question 2

(a)

- **Indexing field (indexing attribute):** A field of a file on which an index access structure is built, used to construct index entries that point to disk blocks or records containing that field's value.
- **Block anchor:** The first record stored in each block of the data file (in some implementations, the last record); its key value is used as the K(i) value in a primary index entry for that block.
- **Dense index:** An index that contains an index entry for *every* search key value (and hence for every record) in the data file.
- **Sparse (nondense) index:** An index that contains index entries for only *some* of the search/key values in the file (fewer entries than there are records) — such as one entry per block rather than one per record.

(b)

- (i) Primary index — **Sparse (nondense)**: it has only one entry for each block of the data file (the block anchor), not one entry per record.
- (ii) Secondary index on a key field — **Dense**: because the field is a key (unique value per record) and the data file is not physically ordered on it, there must be a separate index entry for every record so that each record can be individually located.

Question 3 (10 marks)

Consider a B+-tree of order **p = 5** for internal nodes and **p_leaf = 4** for leaf nodes.

(a) Describe the structure of an **internal node** of this B+-tree: what does it contain, and how many tree pointers and search values can it hold at most? **(4 marks)**

(b) Describe the structure of a **leaf node** of this B+-tree: what does it contain, and how is it different from an internal node? **(4 marks)**

(c) Explain, in one or two sentences, why the leaf nodes of a B+-tree are usually linked together using pointers. **(2 marks)**

Model Answer — Question 3

(a) An internal node of a B+-tree of order p = 5 has the form `<P1, K1, P2, K2, …, Pq-1, Kq-1, Pq>` where q ≤ p. Each Pi is a *tree pointer* to a child node — there are no data pointers in internal nodes. With p = 5, an internal node can hold at most **5 tree pointers** and **4 search key values**.

(b) A leaf node of a B+-tree (with p_leaf = 4) contains entries of the form `<K1, Pr1>, <K2, Pr2>, …` where each Pri is a **data pointer** (to the record or to the block containing the record with that key value), not a tree pointer. With p_leaf = 4, a leaf node can hold up to 3 (or up to p_leaf − 1, depending on the exact convention used) key–data-pointer pairs, plus a pointer to the next leaf node. Unlike internal nodes, leaf node entries point directly to data, not to other tree nodes.

(c) Leaf nodes are linked together (typically left-to-right) so that the records can be retrieved in **ordered sequence** by the search field without having to traverse back up through the internal nodes — this efficiently supports range queries and sequential scans.

Question 4 (8 marks)

(a) What is **partitioned hashing**, and what type of query condition is it suitable for? **(4 marks)**

(b) State the **main limitation** of partitioned hashing and explain why this limitation exists. **(4 marks)**

Model Answer — Question 4

(a) Partitioned hashing is an extension of static external hashing that supports access on **multiple keys**. For a composite key of *n* components, the hash function produces *n* separate hash addresses, and the bucket address is formed by **concatenating** these n addresses. It is suitable only for **equality comparisons** on some or all of the component attributes (e.g., searching for Dno = 4 and Age = 59, or just Age = 59 across all Dno partitions).

(b) The main limitation is that partitioned hashing **cannot support range queries** on any of the component attributes. This is because hash functions, by design, scatter values pseudo-randomly across the address space and do **not preserve the order** of the original key values — so there is no way to identify a contiguous set of bucket addresses corresponding to a range of key values (e.g., Age between 30 and 40), unlike an ordered index or a grid file.

Section C — Essay Questions (40 Marks)

Answer TWO of the THREE questions in this section. Each question carries 20 marks.

Essay Question 1 (20 marks)

Critically compare and contrast **primary indexes, clustering indexes, and secondary indexes** as the three types of single-level ordered indexes. Your essay must:

(a) Define each of the three index types, explicitly stating the nature of the indexing field (key vs. nonkey) and its relationship to the physical ordering of the data file. **(9 marks)**

(b) For each index type, state whether it is dense or sparse, and explain *why* this is structurally necessary given how each index is built. **(6 marks)**

(c) Explain why a data file can have **at most one** primary or clustering index, but can have **several** secondary indexes, linking your explanation to the physical constraints of file storage. **(5 marks)**

Model Answer — Essay Question 1

(a) Definitions:

- A **primary index** is built on the *ordering key field* of an ordered data file — a field that is both used to physically sort the records on disk AND has a unique value for every record (it is a key). There is one index entry per *block* of the data file, containing the key value of the block anchor (the first record in the block) and a pointer to that block.
- A **clustering index** is built on the *ordering field* of an ordered data file when that field is a *nonkey* — i.e., many records may share the same value (e.g., Department_number). The data file in this case is called a clustered file. There is one index entry per *distinct value* of the clustering field, pointing to the first block containing a record with that value.
- A **secondary index** is built on a *nonordering* field of the data file — meaning the field used for the index has no relationship to how the records are physically arranged on disk. It can be built on a key (unique) field or a nonkey field, and a file can have several such indexes simultaneously, each providing an independent access path.

(b) Dense vs. Sparse, and why:

- **Primary index — Sparse (nondense):** Because the data file is physically ordered by the indexing field, all records with values between two consecutive block anchors are guaranteed to reside in the same block. It is therefore only necessary to record one entry per block (the anchor), not one per record — the physical ordering does the rest of the work.
- **Clustering index — Sparse (nondense):** Similarly, because the data file is ordered by the (nonkey) clustering field, all records sharing a given value are physically grouped together, starting at a known block. One entry per *distinct value* is sufficient to locate the start of that group.
- **Secondary index — Dense (when on a key field):** Since the data file is *not* physically ordered by the indexing field, there is no guarantee about where records with similar values are located relative to each other — a record with a given key value could be anywhere in the file. Therefore, an index entry must exist for *every record* to guarantee that any individual record can be found. (Note: secondary indexes on nonkey fields can use various non-dense implementation options with an added level of indirection, but the *key-field* case is necessarily dense.)

(c) Why at most one primary/clustering index but several secondary indexes:
A file's records can be **physically sorted on disk in only one way at a time** — a file has at most one physical ordering field. Since a primary index requires the data file to be ordered by a key field, and a clustering index requires it to be ordered by a nonkey field, only one of these two structural arrangements can exist for the file simultaneously (and never both, since a field cannot be both a key and a nonkey). In contrast, a **secondary index does not require or alter the physical ordering of the data file** — it is a wholly separate, independently-maintained structure layered "on top" of the existing storage order. Because adding a secondary index has no implications for how records are physically arranged, any number of additional secondary indexes can be created on different fields without conflicting with one another or with the primary/clustering index.

Essay Question 2 (20 marks)

B+-trees are described as the dominant access structure used for indexing in modern relational database systems.

(a) Explain the structural rules that define a B+-tree of order *p*, distinguishing clearly between internal nodes and leaf nodes. **(8 marks)**

(b) Using a concrete, original numeric scenario of your own choosing (state your own values for block size, key size, and pointer sizes), calculate the order *p* of a B+-tree and discuss how many levels would be needed to index a data file of your chosen size. **(8 marks)**

(c) Explain why most commercial DBMSs (e.g., Oracle) implement **all** indexes as B+-trees rather than as B-trees. **(4 marks)**

Model Answer — Essay Question 2

(a) Structural rules of a B+-tree of order p:
A B+-tree is a balanced, dynamic multilevel index in which **all leaf nodes are at the same level**, and the structure of leaf nodes differs fundamentally from internal nodes:

- **Internal nodes** have the form `<P1, K1, P2, K2, …, Kq-1, Pq>` where q ≤ p. Every Pi is a *tree pointer* to a child node — internal nodes contain **no data pointers at all**. Their sole purpose is to guide the search down to the correct leaf.
- **Leaf nodes** contain entries of the form `<Ki, Pri>` where each Pri is a **data pointer** — either directly to the record (if the search field is a key) or to a block of record pointers (if it is a nonkey field, via an extra level of indirection). Leaf nodes are typically **linked together** (via a "next" pointer) to support ordered, sequential access along the search field.
- Because internal nodes carry only keys and tree pointers (and not the heavier data pointers), they can pack in **more entries per node** than a B-tree internal node of the same block size, which directly increases the fan-out and reduces the number of levels needed.

(b) Worked numeric example (illustrative values):
Suppose: block size B = 1,024 bytes, search key size V = 9 bytes, block/tree pointer size P = 6 bytes, record pointer size Pr = 7 bytes, and the data file has r = 100,000 records.

- *Internal node order p:* each internal node entry is (P + V) = (6 + 9) = 15 bytes, plus one extra tree pointer, so p is the largest integer such that `p × 6 + (p − 1) × 9 ≤ 1,024` → solving gives p ≈ 68.
- *Leaf node order p_leaf:* each leaf entry is (V + Pr) = (9 + 7) = 16 bytes, plus a "next" pointer (6 bytes), so p_leaf is the largest integer such that `p_leaf × 16 + 6 ≤ 1,024` → p_leaf ≈ 63.
- At ~69% occupancy: internal fan-out ≈ 0.69 × 68 ≈ 47; leaf occupancy ≈ 0.69 × 63 ≈ 43 entries per leaf.
- Number of leaf blocks needed ≈ 100,000 / 43 ≈ 2,326 leaf blocks.
- Level 1 (above leaves) needs ≈ 2,326 / 47 ≈ 50 blocks; Level 2 (top) needs ≈ 50/47 ≈ 2 blocks → effectively 1 root block.
- This gives a **3-level tree** (root → internal level → leaf level), meaning a search requires about **4 block accesses** total (3 index levels + 1 data block, or 3 if the leaf directly stores enough to avoid a separate data access). *(Exact numbers will vary depending on the values a candidate chooses — credit should be given for correct method and consistent arithmetic, not a specific numeric answer.)*

(c) Why B+-trees are preferred over B-trees:
Because B-trees store a **data pointer alongside every key at every level** (including internal nodes), each B-tree internal node entry is larger than the corresponding B+-tree internal node entry (which has no data pointer). For the same block size, this means **B+-tree internal nodes can hold more keys and pointers** than B-tree internal nodes, producing a **higher fan-out** and therefore **fewer levels** for the same number of records — directly translating into fewer block accesses per search. Additionally, B+-trees' linked leaf-level structure naturally supports efficient **sequential and range-based access**, which is heavily used in SQL queries (`BETWEEN`, `ORDER BY`, etc.), making B+-trees the practical default choice in virtually all major RDBMSs.

Essay Question 3 (20 marks)

Physical database design requires the database designer to make a series of structured decisions about storage and access paths, guided by an understanding of the expected "job mix" of queries and transactions.

(a) Explain the FIVE factors that influence physical database design, as identified in the chapter (query/transaction analysis, frequency of invocation, time constraints, frequency of updates, and uniqueness constraints). **(10 marks)**

(b) Using a hypothetical online retail database (e.g., tables for `Customer`, `Order`, and `Product`) as a case study, apply EACH of the five factors from part (a) to justify a specific indexing decision for this database. **(10 marks)**

Model Answer — Essay Question 3

(a) The five factors influencing physical database design:

1. **Analyzing database queries and transactions:** Before any physical design decision, the designer must understand, for each expected query, which files it accesses, which attributes carry selection conditions (and whether these are equality, inequality, or range conditions), which attributes are used in join conditions, and which attributes are merely retrieved. The attributes used in selection and join conditions are the prime *candidates* for indexing. For update operations, the designer must similarly identify which files are updated, the operation type (insert/update/delete), which attributes drive the selection condition for the update/delete, and which attributes have their *values* changed — attributes in this last category are candidates for **avoiding** an index, since modifying an indexed attribute also requires updating the index.

2. **Frequency of invocation:** Not all queries run equally often. The designer must compile the expected frequency with which each attribute is used as a selection or join attribute, across all queries and transactions. The informal **80–20 rule** applies: roughly 80% of processing load comes from just 20% of queries/transactions, so the designer can focus indexing effort on this most-frequent subset rather than exhaustively profiling every possible query.

3. **Time constraints:** Some transactions carry strict performance requirements (e.g., "must complete within 5 seconds 95% of the time, never more than 20 seconds"). Selection attributes used by such time-constrained transactions become **higher-priority candidates** for the most efficient access structures available — typically the primary or clustering index — since these are generally the fastest way to locate records.

4. **Frequency of update operations:** Every index on a file adds overhead to insert, update, and delete operations, because each index must itself be updated whenever the underlying data changes. For files with very frequent insertions, having too many indexes (e.g., 10 indexes on 10 different attributes) can substantially slow down those insert operations — so the designer must limit the number of indexes on heavily-updated files to only those that are truly justified by query performance gains.

5. **Uniqueness constraints:** Access paths (typically indexes) should be created on all candidate key attributes or attribute sets — i.e., the primary key or any UNIQUE-constrained attribute. This is because the existence of an index makes it possible to check the uniqueness constraint by searching *only the index* (since all attribute values appear in its leaf nodes) rather than scanning the entire data file, making uniqueness enforcement during insertion far more efficient.

(b) Application to an online retail database:

- **Query/transaction analysis:** Suppose frequent queries include "find all orders for a given Customer_id" (selection) and "find the product name for a given Order line's Product_id" (join). This identifies `Order.Customer_id` and `Product.Product_id` as strong candidates for indexing, since they appear directly in selection and join conditions respectively.
- **Frequency of invocation:** If 80% of database load comes from the "view my orders" (filter by Customer_id) and "checkout" (insert into Order) operations, design effort should prioritize indexing Customer_id heavily and ensure the Order insert path remains lightweight, rather than spending equal effort optimizing rarely-run administrative reports.
- **Time constraints:** If the "view my orders" query must return in under 2 seconds during checkout flows (a hard UX requirement), `Customer_id` on the `Order` table becomes a high-priority candidate for a **clustering index** (since many orders share the same Customer_id, it is a nonkey ordering field), as this is the most efficient access structure for retrieving all of a customer's orders together.
- **Frequency of updates:** If the `Order` table receives thousands of new rows per minute (very frequent inserts) but rarely has existing rows updated, the designer should be cautious about creating too many secondary indexes on `Order` (e.g., avoid indexing low-value, rarely-queried columns like `internal_notes`), since each additional index adds insertion overhead to an already high-throughput table.
- **Uniqueness constraints:** Since `Product.Product_id` and `Customer.Customer_id` are primary keys (unique by definition), each must have an associated index (typically automatically created by the DBMS as a primary index) — this both speeds up lookups and allows the DBMS to efficiently reject duplicate-key insert attempts by checking only the index rather than scanning the full table.

End of practice exam. Recommended approach: complete Section A in 15 minutes, Section B in 75 minutes (about 18–19 minutes per question), and Section C in 75–90 minutes for your two chosen essays (37–45 minutes each).